THE XTREAM MULTIMEDIA SYSTEM

Zoran Dimitrijević *Raju Rangaswami* *Edward Chang*

University of California, Santa Barbara

ABSTRACT

This paper presents the architecture and implementation of XTREAM, a high-performance streaming multimedia system. High performance in XTREAM is enabled by its three core components: *IO Scheduler*, *Request Handler*, and *Admission Controller*. By conducting extensive experiments, we demonstrate that thanks to the core components, XTREAM can achieve high throughput while offering low response time and guaranteeing quality of service to multimedia clients.

1. INTRODUCTION

Traditional operating systems are mostly optimized for interactive or throughput intensive IO workload. However, multimedia systems place unique, real-time demands on disk performance. In multimedia systems, data must be retrieved from disk and played back by a specific deadline. If the deadline is not met, end users experience unacceptable video jitters or audio pops. A multimedia user also expects fast access to content which translates to a low initial latency requirement for storage access. Thus, streaming multimedia presents the conflicting requirements of guaranteed delivery, high throughput and short initial latency to a multimedia system. For example, reducing the size of disk requests we can reduce the initial latency and the required memory buffer. However, the achieved disk-throughput utilization degrades, and the system cannot support a large number of streams.

In this paper, we describe the implementation of XTREAM, a streaming multimedia system, which supports the above requirements, namely:

1. High throughput

2. Low initial latency

3. Guaranteed IO

To guarantee high throughput, XTREAM uses an *IO Scheduler* module for servicing disk IOs. To offer low initial latency, a *Request Scheduler* component serves new requests with high priority. To guarantee quality of service (QoS) to multimedia streams, XTREAM employs an *Admission Controller* module which ensures that all IO requests can be completed in time and that the system is not under-utilized. The design of the three components of XTREAM is based on accurate disk drive modeling, using our disk profiling tool [1].

XTREAM runs in user mode, and supports heterogenous streaming media types (with different bit-rates) as well as non-real-time data retrieval. It supports guaranteed-rate IO for both write (e.g., recording by a surveillance camera) and read streams (e.g., mp3 or video playback). XTREAM can also effectively deal with variable bit-rate (VBR) streams (e.g., compressed video). XTREAM scheduler inherently supports servicing non-real-time data like pure-text or html without violating real-time streaming requirements. It can also provide differentiated services since it supports multiple QoS levels.

The rest of this paper is organized as follows: In Section 2, we present the design of XTREAM. In Section 3, we evaluate XTREAM by extensive experimentation and subjecting it to severe overload conditions. We present related work in Section 4 and suggest directions for future work in Section 5.

2. SYSTEM DESIGN

In this section, we provide an overview of the design of the XTREAM system. The service model consists of one or more XTREAM clients connecting to an XTREAM server to request for multimedia data stored on the server's disk drive. The client could be desktop requesting a video-on-demand service, a surveillance camera recording video, a TV-based client requesting delayed playback of a TV show, or simply a web-browser requesting html data. The only assumption we make in our model is that the interconnection network between the server and client is not a performance bottleneck.

The architecture of the XTREAM system is presented in Figure 1. The XTREAM client consists of a *proxy* component which connects to the server on behalf of the client application and also performs data buffering to mask network bandwidth variations. The client is designed in such a fashion so that it can easily operate with any encoder or decoder application that supports a UNIX pipe-like interface. The XSTREAM server services IOs for existing streams and decides if it can admit new requests.

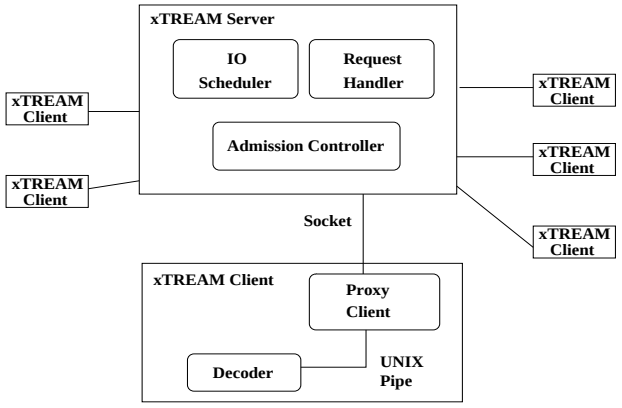


Figure 1: XTREAM system architecture.

The three requirements of XTREAM are *high throughput*, *low initial latency*, and *guaranteed IO*. These three requirements are addressed by the three components in XTREAM, namely the *IO Scheduler* which uses the time cycle model [2] for servicing disk IOs, the *Request Handler* which preempts the IO scheduler for servicing new requests with minimum delay, and the *Admission Controller* which guarantees QoS for soft-real-time multimedia requests while ensuring non-starvation of non-real-time data retrievals. In addition to these components, XTREAM uses a *Disk Profiler* [1] in order to get realistic disk model required to predict disk performance and provide real-time streaming guarantees.

2.1. IO Scheduler

XTREAM adopts a single-thread IO paradigm wherein the IO scheduler consists of a single thread performing all disk IOs. It uses the popular time cycle model [2], in which time is split into basic units called time cycles ($\mathcal{T}$). In a single time cycle, XTREAM services exactly one disk IO per stream. In each cycle, the size of the IO is chosen so that the display buffer does not underflow before the next IO is performed. The only deviation from the original time cycle model is that we allow the scheduling order to vary between cycles by using a *double buffer* for each stream which can sustain playback for as much as two time cycles. This is essential to ensure low initial latency for user requests. In contrast, the simple multi-threaded approach wherein each multimedia stream is served by a dedicated thread is found lacking. The single thread IO paradigm performs better due to the following reasons:

Deterministic Execution: Since a single thread is performing all disk IOs, the IO schedule is deterministic. This deterministic behavior enables incorporating IO guarantees for individual streams. In the multi-threaded IO model, the OS thread scheduling determines the IO order and there is no bound on when any requested IO will be completed.

Controlled IO variability: IO variability is defined as the variability in time between successive IOs for the same stream. Large IO variability requires more in-memory buffering and increases the system cost. The single-thread model controls IO variability by performing at least one IO for each stream in each cycle. This is not possible in the multi-threaded design.

Contiguous Reads: Since the operating system might break a large IO request into multiple small ones, a single read operation might incur multiple disk accesses simply due to thread switching in a multi-threaded design. Blocking IO used in the single-threaded design ensures that a read operation to the disk is indeed sequential.

Fairness: In the single-thread IO model, it is easy to incorporate service for non-real-time request by simply reserving a fixed portion of each cycle (fixed disk-bandwidth, in other words) to service non-real-time jobs. In contrast, non-real-time jobs could potentially be starved under the multi-threaded model in the absence of aged priority increase in the operating system.

2.2. Request Handler

When a new request arrives in the XTREAM system, the request handler module is invoked to service it. The request handler, in turn, invokes the admission controller module to determine if the new request can be granted service or not. If not, the request handler denies the request. If the request can be admitted, the request handler preempts the IO scheduler module as soon as it finishes servicing the current IO request. It then adds the request to the head of the IO service queue, which is used by the IO scheduler to determine the service order. We can make two observations for this approach:

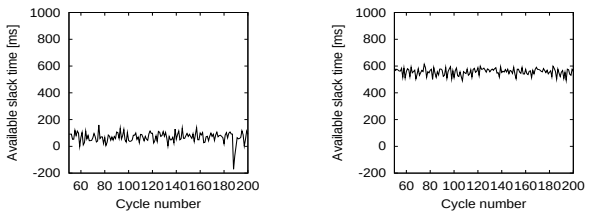
1. This approach offers an initial latency bound which does not depend on the number of streams in the system. The initial latency is simply the sum of the maximum time required to service a single IO for any existing stream and the time required to perform the initial IO for filling up the buffer of the new stream. The initial latency can be further improved by preempting the current disk IO using our implementation of preemptible disk access [3].

2. This approach comes at the cost of using double buffering which provides freedom from maintaining the same IO schedule between time cycles. However, the double buffering scheme offers the additional freedom from using *fixed-stretch* [2], in which the IO for a stream must be started exactly at the same time relative to the beginning of each cycle. In a system which serves both soft-real-time multimedia requests and non-real-time requests, a fixed-stretch IO restriction might lead to under-utilization of disk bandwidth. Hence, we believe that the choice for double-buffering is a practical one for a system that supports heterogenous request types.

2.3. Admission Controller

One of the design goals of XTREAM is to provide real-time guarantees for multimedia streams. To ensure that the server is not overloaded, XTREAM employs an *Admission Controller* module that decides if a new stream can be admitted or not without violating QoS requirements for existing streams. The main objectives of the Admission Controller are: (1) maintaining QoS and (2) avoiding under-utilization of the system.

Figure 2 shows available *slack* in each time cycle for two scenarios. If the available slack is always greater than zero, then all deadlines are fulfilled, and the system is able to support all streams in real-time. Figures 2(a) and 2(b) illustrate the variations of available slack when our system is slightly overloaded (i.e., cannot maintain real-time guarantees) and under-utilized (i.e., can admit more streams) respectively. In order to



(a) Overloaded server (b) Under-utilized server

Figure 2: Available slack in each time cycle.

achieve the two design objectives, XTREAM has to be able to

predict disk-throughput utilization accurately. This is a challenging problem because the disk performance varies significantly depending on the disk access pattern. The Admission Controller in XTREAM does not make any assumptions on file-system data layout on the disk or any other knowledge about file-system placements. The only assumption made by the Admission Controller is that a single IO is sequential. This is a reasonable assumption for multimedia files where the ratio of the file size to the IO size is large. This feature of XTREAM allows it to work with almost any file-system that the user may choose to employ.

Equation 1 illustrates simple model for disk utilization (U) which depends on the number of IO requests in one cycle (N). Transfer time $T_{transfer}$ is the total time disk spends in data transfer from disk media. Access time T_{access} is the average access time penalty for each IO request and includes both disk seek time and rotational delay.

$$U = \frac{T_{transfer}}{N \times T_{access} + T_{transfer}} \quad (1)$$

Since the disk utilization U depends only on the number of requests and the total amount of transfered data, it can be expressed as a function of just one parameter: the average IO request size (S_{avg}). We use our disk profiler tool to find out

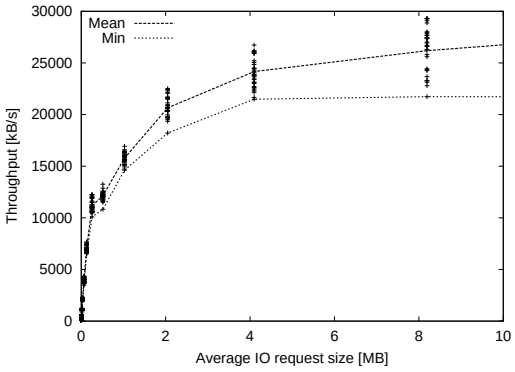


Figure 3: Achieved disk throughput vs. average IO size.

disk-throughput utilization of an actual disk. The profiler performs sequential reads of the same size from random positions on the disk. Figure 3 shows achieved disk throughput depending on the average IO request size (which is equal to the size of random sequential reads). In this paper, we propose and evaluate two classes of approaches for admission control: (1) *conservative* and (2) *aggressive*. The conservative class provides the best QoS level for all streams, while the aggressive class provides support for tunable QoS levels.

Let the bit-rate of each stream i in the system be denoted by BR_i. When new request comes (with required bit-rate BR_{new}), the Admission Controller module calculates the required new average IO request size using Equation 2.

$$S_{avg} = \frac{\mathcal{T} \times (BR_{new} + \sum_{i=1}^{N} BR_i)}{N} \quad (2)$$

In the next step, we obtain the predicted disk utilization, $P(S_{avg})$, for an average request size of S_{avg} from the disk utilization graph (Figure 3). Then, Equation 3 decides it new stream can be admitted. If condition holds, the new request is accepted. Otherwise, the client request is denied.

$$P(S_{avg}) > BR_{new} + \sum_{i=1}^{N} BR_i \quad (3)$$

3. RESULTS

In this section we evaluate the XTREAM system using the following metrics: 1) maximum system throughput, 2) initial latency, and 3) accuracy of admission control methods. The system is evaluated using Intel Pentium 4 1.5 GHz Linux based PC, with 512 MB of main memory and a WD400BB 40 GB hard drive. Maximum sequential disk throughput is 31 MBps in the fastest zone and 21 MBps in the slowest zone. The LAN is 100 Mbps ethernet which enables streaming of several MPEG2 and large number of MPEG4 encoded videos.

In order to evaluate hard disk scheduler, client can require "dummy" streaming with constant (CBR) or variable bit-rate (VBR). Dummy streams are not streamed over the network. The client can specify if the dummy stream is read or write, its file name, and the required constant or variable bit-rate.

3.1. Throughput

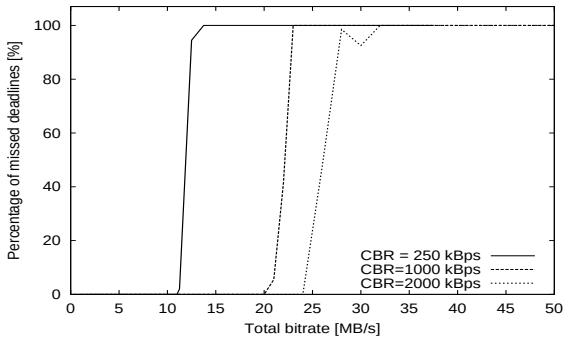


Figure 4: Percentage of missed cycle deadlines.

Figure 4 shows the percentage of missed cycle deadlines depending on the total bit-rate of serviced streams. Since one of the main goals of the system is to maintain real-time guarantees, the maximum throughput of the system is the maximum value on x-axis where the system does not experience any missed deadlines. Depending on the admission control used, our system will admit less number of streams and effectively reduce the maximum throughput. These results are summarized in Table 1, where the maximum system throughput can be obtained by multiplying the number of admitted streams with the corresponding average bit-rate.

3.2. Initial Latency

In this paper we define *initial latency* as the time between the moment when Request Handler receives client request, and the moment when buffer for the new stream is filled. We do not

consider and evaluate network and client-side latency. The results for initial latency depending on the number of streams in the system, are presented in Figure 5. Initial latency does not depend on the load of the system but only on the size of IO requests, as explained in Section 2.

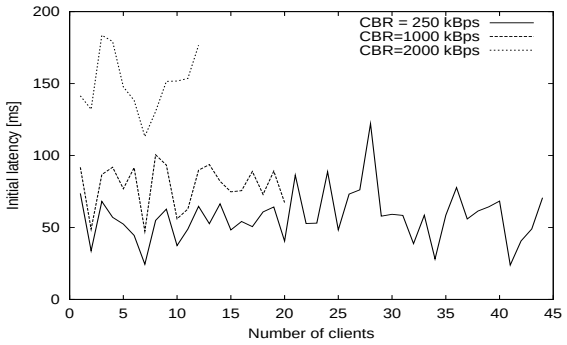


Figure 5: Initial latency.

3.3. Guaranteed IO

In this section, we evaluate three different admission control configurations introduced in Section 2.3: conservative admission control ($Adm1$) and two aggressive admission controls ($Adm2$ and $Adm3$). $Adm1$ uses the min curve from Figure 3. $Adm2$ uses the mean curve from Figure 3 and maximum stream bit-rates. $Adm3$ uses mean curve from Figure 3 and average stream bit-rates.

Avg. BR	Type	$MaxN$	$Adm1$	$Adm2$	$Adm3$
250 kBps	C	44	39	43	n/a
1000 kBps	C	20	14	15	n/a
2000 kBps	C	12	9	10	n/a
250 kBps	V	44	30	34	43
1000 kBps	V	23	11	12	15
2000 kBps	V	11	7	8	10
250 kBps	H	44	39	43	n/a
1000 kBps	H	16	14	15	n/a
2000 kBps	H	10	9	10	n/a

Table 1: System throughput.

Table 1 summarizes the accuracy of the evaluated admission control configurations. We performed experiments for the following scenarios. Homogeneous CBR (type C) and VBR (type V) streams where all serviced streams have the same average bit-rate. Heterogenous CBR streams (type H) where each stream has a constant bit-rate, but the bit-rate varies between $1/10$ and 10 times the average value. $MaxN$ is the maximum number of streams that the system can support without missing deadlines. Based on Table 1, we believe that our system is not severely under-utilized while providing guaranteed-rate IO.

4. RELATED WORK

Multimedia file-system efforts in recent years include [4, 5, 6, 7, 8, 9]. Of these, the industry initiatives usually do not disclose their implementations. To the best of our knowledge, existing admission controllers for streaming multimedia systems do not use disk modeling based on low-level disk profiling.

5. CONCLUSION

In this paper, we presented the design and implementation of the XTREAM multimedia streaming infrastructure. High performance in XTREAM is achieved by its three core components: *IO Scheduler*, *Request Handler*, and *Admission Controller*. In addition to these, XTREAM uses a *Disk Profiler* to accurately estimate the performance of the hard-disk. We plan to further our research in two directions. First, we plan to investigate the importance of preemptible disk access [3] in improving initial latency of our scheduler. Second, we plan to implement our scheduler in Linux kernel and then investigate different admission control strategies which are possible in the kernel space.

6. REFERENCES

[1] Zoran Dimitrijevic, Raju Rangaswami, Edward Chang, David Watson, and Anurag Acharya, "Diskbench," *http://www.cs.ucsb.edu/∼zoran/diskbench/*, November 2001.

[2] Edward Chang and Hector Garcia-Molina, "Effective memory use in a media server," *Proceedings of the 23rd VLDB Conference*, pp. 496–505, August 1997.

[3] Zoran Dimitrijevic, Raju Rangaswami, and Edward Chang, "Virtual IO: Preemptible disk access (extended version)," *http://www.cs.ucsb.edu/∼zoran/papers/vio02x.pdf*, April 2002.

[4] R. Haskin and F. Schmuck, "The tiger shark filesystem," *IEEE COMPCON*, 1996.

[5] M. Holton and R. Das, "Xfs: A next generation journalled 64-bit filesystem with guaranteed rate IO," *SGI Technical Report*, 1996.

[6] C. Martin, P. Narayan, B. Ozden, and R. Rastogi, "The fellini multimedia storage system," *Journal of Digital Libraries*, 1997.

[7] A. Molano, K. Juvva, and R. Rajkumar, "Guaranteeing timing constraints for disk accesses in RT-Mach," *Real Time Systems Symposium*, 1997.

[8] Prashant Shenoy, Pawan Goyal, S. S. Rao, and Harrick Vin, "Symphony: An integrated multimedia file system," *Proceedings of the Multimedia Computing and Networking (MMCN)*, 1998.

[9] Vijay Sundaram, Abhishek Chandra, Pawan Goyal, Prashant Shenoy, J. Sahni, and Harrick Vin, "Application performance in the qlinux multimedia operating system," *ACM Multimedia*, 2000.